Algoritmos de Busca

Aula 06 – Sequencial, Binária e por Espalhamento (Hash)



C.E.S.A.R

@CESAR 2021| Todos os Direitos Reservados

Algoritmos de Busca

- **Buca Sequencial;**
- **Busca Binária;**
- **Busca por Espalhamento (Hash);**

@CESAR 2021| Todos os Direitos Reservados

Busca Sequencial

Busca Sequencial

@CESAR 2021| Todos os Direitos Reservados

- A função deve receber uma lista de números inteiros e uma chave de busca;
- A função deve retornar o índice da lista que contém a chave ou o valor -1, caso a chave não esteja na lista;
- A busca sequencial é a mais simples, útil para conjuntos pequenos ou não ordenados;
- Consiste em verificar elemento a elemento até encontrar o valor desejado;
- Na melhor das hipóteses, a chave de busca estará na posição 0. Na pior das hipóteses, a chave é o último elemento ou não pertence a lista e portanto, acessamos todos os elementos da lista.
- É possível mostrar que, se as chaves possuírem a mesma probabilidade de serem requisitados, o número médio de acessos nas buscas cujas chaves encontram-se na lista será igual a:

(N + 1) / 2

Valor	30	15	20	55	8	10
Índice	0	1	2	3	4	5

Busca Sequencial

@CESAR 2021| Todos os Direitos Reservados

```
#include <iostream>

int buscaSequencial(int vetor[], int tamanho, int valor) {
    for (int i = 0; i < tamanho; i++) {
        if (vetor[i] == valor)
            return i; // retorna índice se encontrado
    }
    return -1; // não encontrado
}

int main() {
    int vetor[] = {4, 8, 15, 16, 23, 42};
    int valor = 15;
    int pos = buscaSequencial(vetor, 6, valor);

    if (pos != -1)
        std::cout << “Valor: “ << valor << “ encontrado na posicao: “ << pos << “.\n”;
    else
        std::cout << “ Valor nao encontrado.\n”;

    return 0;
}
```

@CESAR 2021| Todos os Direitos Reservados

Busca Binária

Busca Binária

@CESAR 2021| Todos os Direitos Reservados

- Busca binária deve ser usado em listas ordenadas, dividindo sucessivamente a lista ao meio até encontrar o valor.
- Como funciona?

1. Verifique se a chave de busca, é igual ao valor da posição do meio da lista;
2. Caso o valor desta posição seja maior do que a chave, então repita o processo, mas considere uma lista reduzida com os elementos do começo da lista até a posição anterior a do meio;
3. Caso o valor desta posição seja menor que a chave, repita o processo, uma lista reduzida, com os elementos da posição seguinte a do meio até o final da lista.

- Na melhor das hipóteses, a chave da busca estará na posição do meio da lista. Portanto, teremos um único acesso. Na pior das hipóteses, dividimos a lista até que ela fique com um único elemento (último acesso realizado à lista).
- Note que, a cada acesso, o tamanho da lista é diminuído, pelo menos, pela metade;
- É possível mostrar que, se as chaves possuírem a mesma possibilidade de serem requisitadas, o número médio de acessos nas buscas cujas chaves encontram-se na lista será igual a:

(log n) - 1

Comparação Sequencial x Binária

@CESAR 2021| Todos os Direitos Reservados

- Para se ter uma ideia da diferença de eficiência dos dois algoritmos, considere uma lista com um milhão de itens (10^6 itens).
- Com a busca Sequencial, para buscar um elemento qualquer da lista necessitamos, em média, de:

$$((10^6 + 1) / 2) \approx 500.000$$

- Com a busca Binária, para buscar um elemento qualquer da lista necessitamos, em média, de:

$$\log 10^6 \approx 1 \sim 19$$

Busca Binária

@CESAR 2021| Todos os Direitos Reservados

```
#include <iostream>

int buscaBinaria(int vetor[], int tamanho, int valor) {
    int inicio = 0, fim = tamanho - 1, meio;

    while (inicio <= fim) {
        meio = (inicio + fim) / 2;

        if (vetor[meio] == valor)
            return meio; // valor encontrado
        else if (vetor[meio] < valor)
            inicio = meio + 1; // busca na metade superior
        else
            fim = meio - 1; // busca na metade inferior
    }
    return -1; // não encontrado
}
```

```
int main() {
    int vetor[] = {3, 6, 9, 12, 15, 18, 21};
    int valor = 15;
    int pos = buscaBinaria(vetor, 7, valor);

    if (pos != -1)
        std::cout << "Valor: “ << valor << “ encontrado na
posicao: “ << pos << “.\n”;
    else
        std::cout << "Valor nao encontrado.\n";

    return 0;
}
```

@CESAR 2021| Todos os Direitos Reservados

Espalhamento (Hash)

@CESAR 2021| Todos os Direitos Reservados

Espalhamento (Hash)

- A busca por espalhamento (Hashing) usa uma função hash para transformar valores em índices, permitindo buscas rápidas em tempo constante O(1).
- Ao inserir um elemento, realiza um calculo, onde retornará um índice, e após isso realizará uma inserção.
- Para realizar a busca do valor desejado, basta usar a mesma função de calculo de inserção, dessa maneira, realizando apenas uma comparação.

CHAVE	VALOR
	0
	1
BIA	2
	3
PAULA	4
CARLOS	5

- Cuidado com as colisões!!!!
- Temos várias maneiras de tratar as colisões:
- Lista Encadeada (externo);
- Colocar no próximo espaço livre (interno).

@CESAR 2021| Todos os Direitos Reservados

Espalhamento (Hash) Simples

```
#include <iostream>
#define TAM 10

int tabelaHash[TAM];

// Função hash simples
int hash(int chave) {
    return chave % TAM;
}

// Inserir elemento
void inserir(int chave) {
    int indice = hash(chave);
    tabelaHash[indice] = chave;
}

// Buscar elemento
int buscar(int chave) {
    int indice = hash(chave);
    if (tabelaHash[indice] == chave)
        return indice; // encontrado
    else
        return -1; // não encontrado
}
```

```
int main() {
    // Inicializar tabela com -1
    for (int i = 0; i < TAM; i++)
        tabelaHash[i] = -1;

    inserir(15);
    inserir(25);
    inserir(35);

    int valor = 25;
    int pos = buscar(valor);

    if (pos != -1)
        std::cout << "Valor: “ << valor << “ encontrado na posicao hash: “ << pos << “ .\n”;
    else
        std::cout << "Valor nao encontrado na tabela hash.\n";

    return 0;
}
```

@CESAR 2021| Todos os Direitos Reservados

Resumo Comparativo das Buscas

Algoritmo	Complexidade	Ordenação necessária?	Vantagem principal	Principal desvantagem
Busca Sequencial	O(n)	Não	Simplicidade	Lentidão com grandes listas
Busca Binária	O(log n)	Sim	Velocidade	Requer dados ordenados
Busca Hash (ideal)	O(1)	Não	Velocidade altíssima	Tratamento complexo de colisões

@CESAR 2021| Todos os Direitos Reservados

Espalhamento (Hash) com Sondagem Linear (espaço vago)

Nesse método, quando ocorre uma colisão, procura-se sequencialmente na tabela o próximo espaço livre.

Vantagens:
- Boa eficiência em memória (não requer ponteiros ou estruturas adicionais).
- Aproveitamento eficiente do espaço da tabela.

Desvantagens:
- Pode ocorrer o problema de agrupamento ("clustering"), causando lentidão em buscas e inserções posteriores.
- Desempenho piora rapidamente conforme a tabela fica mais cheia (geralmente acima de 70% ocupada).
- Dificuldade adicional para remover elementos, exigindo estratégias como marcar posições removidas.

Espalhamento (Hash)
Sondagem Linear

@CESAR 2021| Todos os Direitos Reservados

```
#include <iostream>

#define TAM 10
#define VAZIO -1
#define REMOVIDO -2

int tabelaHash[TAM];

// Inicializa a tabela
void inicializarTabela() {
    for (int i = 0; i < TAM; i++)
        tabelaHash[i] = VAZIO;
}

// Função hash simples
int hash(int chave) {
    return chave % TAM;
}
```

```
// Inserção com sondagem linear
int inserir(int chave) {
    int indice = hash(chave);

    for (int i = 0; i < TAM; i++) {
        int tentativa = (indice + i) % TAM;

        if (tabelaHash[tentativa] == VAZIO || tabelaHash[tentativa] ==
REMOVIDO) {
            tabelaHash[tentativa] = chave;
            return tentativa; // inserido com sucesso
        }
    }
    return -1; // tabela cheia
}

// Busca com sondagem linear
int buscar(int chave) {
    int indice = hash(chave);

    for (int i = 0; i < TAM; i++) {
        int tentativa = (indice + i) % TAM;

        if (tabelaHash[tentativa] == VAZIO)
            return -1; // não encontrado (posição vazia indica que nunca
foi inserido)

        if (tabelaHash[tentativa] == chave)
            return tentativa; // encontrado!
    }
    return -1; // não encontrado após percorrer toda a tabela
}
```

```
// Remoção com sondagem linear
int remover(int chave) {
    int pos = buscar(chave);

    if (pos != -1) {
        tabelaHash[pos] = REMOVIDO;
        return 1; // removido com sucesso
    }
    return 0; // elemento não encontrado
}

// Impressão da tabela hash
void imprimirTabela() {
    std::cout << "Tabela Hash:\n";
    for (int i = 0; i < TAM; i++) {
        std::cout << ": “ << i;
        if (tabelaHash[i] == VAZIO)
            std::cout << "VAZIO\n";
        else if (tabelaHash[i] == REMOVIDO)
            std::cout << "REMOVIDO\n";
        else
            std::cout << tabelaHash[i] << “\n”;
    }
}
```

Espalhamento (Hash)
Sondagem Linear

@CESAR 2021| Todos os Direitos Reservados

```
int main() {
    inicializarTabela();

    inserir(15);
    inserir(25);
    inserir(35); // colisões com 15 e 25
    inserir(7);
    inserir(17); // colisão com 7

    imprimirTabela();

    int busca = 25;
    int pos = buscar(busca);
    if (pos != -1)
        std::cout << "\nElemento “ << busca << “ encontrado na posicao: “ << pos << “\n";
    else
        std::cout << "\nElemento “ << busca << “ nao encontrado\n";

    remover(25);
    std::cout << "\nTabela apos remover o valor 25:\n";
    imprimirTabela();

    return 0;

}
```

@CESAR 2021| Todos os Direitos Reservados

Espalhamento (Hash) com Lista Encadeada

Nesse método, cada posição da tabela hash possui uma lista ligada que armazena todos os elementos que colidiram naquela posição.

Vantagens:

- Não há restrição quanto ao número de elementos (não sofre com o problema da tabela ficar "cheia").
- Facilidade para lidar com muitas colisões (boa escalabilidade).
- Inserções e remoções são relativamente simples.

Desvantagens:

- Maior uso de memória devido à necessidade de ponteiros.
- Em caso de muitas colisões, o desempenho pode decair de O(1) (ideal) para próximo de O(n).

@CESAR 2021| Todos os Direitos Reservados

Espalhamento (Hash) com Lista Encadeada

```
#include <iostream>
#include <stdlib.h>

#define TAM 10

typedef struct No {
    int chave;
    struct No *prox;
} No;

No* tabelaHash[TAM];

// Inicializa tabela hash
void inicializarTabela() {
    for (int i = 0; i < TAM; i++)
        tabelaHash[i] = NULL;
}

// Função hash
int hash(int chave) {
    return chave % TAM;
}
```

```
/* Inserção com tratamento de colisões por
encadeamento */
void inserir(int chave) {
    int indice = hash(chave);

    No* novo = (No*)malloc(sizeof(No));
    novo->chave = chave;
    novo->prox = tabelaHash[indice];
    tabelaHash[indice] = novo;
}

// Busca elemento na tabela hash
No* buscar(int chave) {
    int indice = hash(chave);
    No* atual = tabelaHash[indice];

    while (atual != NULL) {
        if (atual->chave == chave)
            return atual;
        atual = atual->prox;
    }
    return NULL;
}
```

```
// Impressão da tabela hash
void imprimirTabela() {
    for (int i = 0; i < TAM; i++) {
        printf("[%d]: ", i);
        No* atual = tabelaHash[i];
        while (atual) {
            printf("%d -> ", atual->chave);
            atual = atual->prox;
        }
        printf("NULL\n");
    }
}
```

@CESAR 2021| Todos os Direitos Reservados

Espalhamento (Hash) com Lista Encadeada

```
int main() {

    inicializarTabela();

    inserir(12); inserir(22); inserir(32); inserir(42);
    inserir(15);

    imprimirTabela();

    int chaveBusca = 22;
    No* resultado = buscar(chaveBusca);

    if (resultado)
        std::cout << “Elemento: “ << resultado->chave << “ encontrado na tabela hash.\n";
    else
        std::cout << "Elemento nao encontrado na tabela hash.\n";

    return 0;

}
```

@CESAR 2021| Todos os Direitos Reservados

Quem é o mais Eficiente? Depende!!!

Cenário	Encadeamento (Listas) ✅	Endereçamento Aberto (Sequencial) ✅
Tabela muito grande com fator de carga alto	✔ Melhor	✘ Ruim (clusterização)
Restrições severas de memória	✘ Usa mais memória	✔ Usa menos memória
Remoções frequentes	✔ Fácil de remover elementos	✘ Complicado (requer marcação)
Simplicidade de implementação	✘ Mais complexa (ponteiros)	✔ Mais simples
Tempo de busca estável	✔ Mais previsível	✘ Pode variar muito

@CESAR 2021| Todos os Direitos Reservados

Resumo Geral

- **Encadeamento (Listas)** é geralmente preferível quando a tabela hash será utilizada intensamente (com muitas colisões esperadas ou grande quantidade de elementos), especialmente em sistemas onde a memória não é uma limitação severa.

- **Endereçamento Aberto (Sequencial)** é adequado para cenários onde economia de memória é importante, o fator de carga é baixo ou moderado (até cerca de 70%) e quando a simplicidade e o desempenho em pequenas escalas são prioritários.

Em geral, o **Encadeamento (Listas)** costuma ser mais eficiente e robusto em aplicações do mundo real, principalmente por garantir uma performance mais estável à medida que a tabela cresce ou quando ocorre grande quantidade de colisões.

C . E . S . A . R

Pessoas impulsionando inovação.

Inovação impulsionando negócios.

